ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

ИТМО

Отчёт по лабораторной работе № 3

по дисциплине “Инженерия данных “

Семестр 5

Выполнил студент:
Мавров Артём Николаевич
гр. J3313
ИСУ 466574

Отчет сдан: 30.10.2026

Санкт-Петербург

2026

Содержание

1 Цель и постановка задачи 3

2 Развёртывание объектного хранилища 3

3 Загрузка данных в бакет hh-lake 3

4 Условие варианта 4

5 Подсчёт навыков: собственный MapReduce 5

6 Подсчёт навыков: DuckDB 7

7 Верификация результатов 7

8 Результаты: навыки, пары и тройки 8

9 Влияние навыков на заработную плату 9

10 Региональный анализ 12

11 Профилирование MapReduce и DuckDB 14

12 Бонус: оптимизация расчётов 16

13 Выводы 17

Цель и постановка задачи

Цель работы — построить аналитический пайплайн над объектным хранилищем: поднять RustFS, загрузить в бакет `hh-lake` вакансии своего варианта, посчитать частоты навыков, пар и троек навыков двумя независимыми способами (собственный MapReduce, читающий объект из S3 чанками, и SQL в DuckDB), сверить результаты, оценить влияние сочетаний навыков на зарплату, сравнить три региона, профилировать оба подхода по времени и памяти и оптимизировать расчёты (бонус).

Вариант 9 — «Программист, разработчик на мобильных платформах».

Исходные данные: 10 000 вакансий hh.ru в формате API (по одному JSON-файлу на вакансию). По согласованию взяты все 10 000: в случайной тысяче оказалось бы около 17–20 мобильных вакансий и лишь 3–4 с зарплатой, на которых анализ зарплат и регионов не имеет смысла.

Стенд: Intel Core i5-12500H (12 ядер, 16 потоков), 16 ГБ ОЗУ, Windows 11 Pro, Docker 28.4 (WSL2), RustFS 1.0.4, Python 3.12, DuckDB 2.0.0, boto3, psutil.

Репозиторий с кодом (пакет `hhlake` и SQL-скрипты в `sql/`) и отчётом: https://github.com/artemmavrov/hh-mobile-skills-lake

Развёртывание объектного хранилища

RustFS запущен в Docker по схеме из домашнего задания № 2: один узел и четыре папки-диска `disks/disk1`…`disk4`, собранные в один erasure set переменной окружения

`RUSTFS_VOLUMES=/data/disk{1...4}`

Порт 9001 на машине занят другим проектом, поэтому S3 API проброшен на `localhost:9100`, а консоль — на `9101`. Папки лежат на одном физическом диске, поэтому включён тестовый флаг `RUSTFS_UNSAFE_BYPASS_DISK_CHECK`.

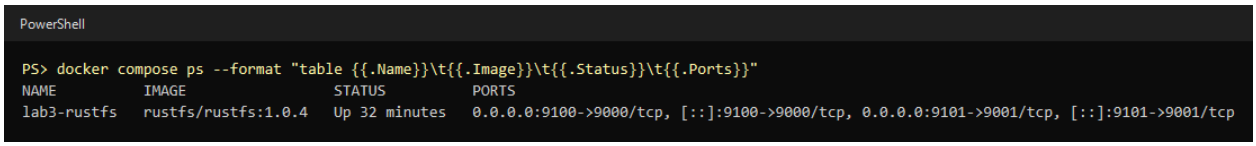


Рис. 1: Контейнер RustFS запущен, S3 API доступен на порту 9100, консоль — на 9101.

Загрузка данных в бакет hh-lake

Скрипт `hhlake/load_data.py` упаковывает 10 000 JSON-файлов в один NDJSON (одна вакансия на строку, порядок по имени файла), создаёт бакет `hh-lake` и загружает объект `raw/hh_vacancies.ndjson` размером 54 994 037 байт. boto3 загружает его multipart-частями по 8 МиБ, отсюда суффикс `-7` в ETag.

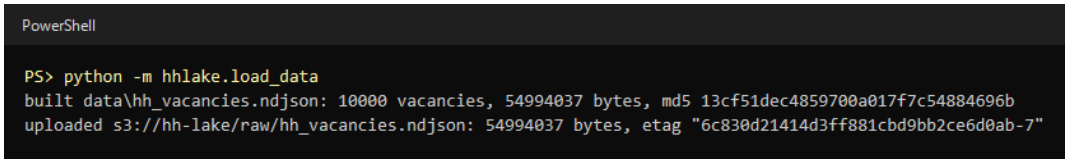


Рис. 2: Сборка NDJSON и загрузка объекта в `s3://hh-lake/raw/`.

Итоговое содержимое бакета после всех запусков приведено на рисунке ниже. Каталог `results/profile/` с результатами профилировочных прогонов скрыт фильтром, чтобы листинг поместился, но строки `Total` учитывают все объекты. Независимую проверку выполняет AWS CLI в контейнере (`scripts/s3.ps1`).

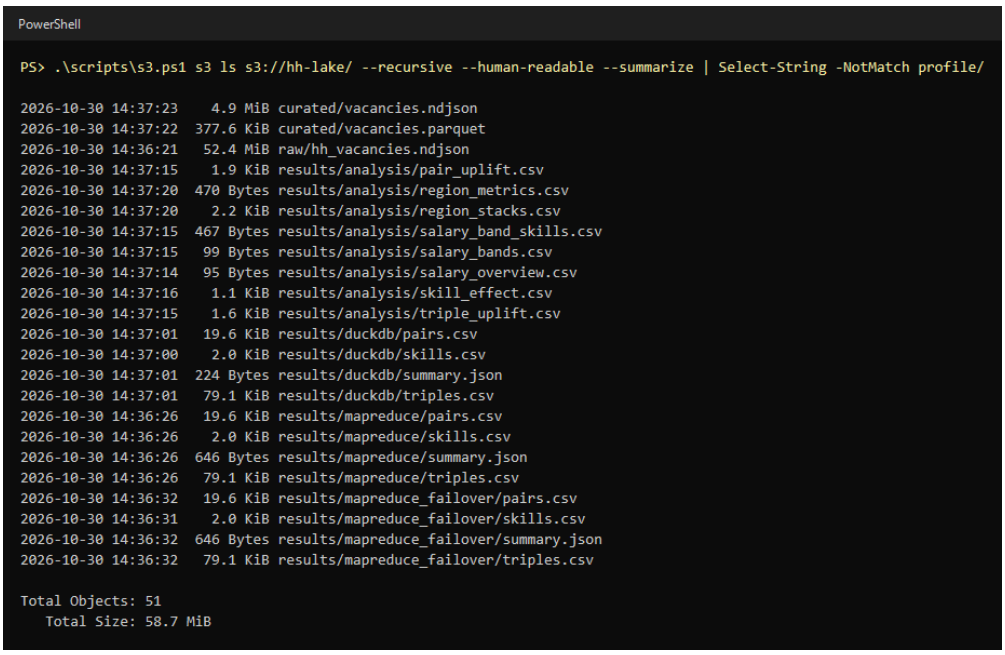


Рис. 3: Бакет `hh-lake`: сырые данные, curated-слой, результаты MapReduce и DuckDB для навыков, пар и троек, прогон с отказом мапперов, аналитические таблицы.

Условие варианта

Вакансия относится к варианту 9, если одновременно выполнены три условия:

1. профессиональная роль — «Программист, разработчик» (id 96) или «Руководитель группы разработки» (id 104);
2. название содержит признак мобильной платформы: Android, iOS, Swift, Mobile, «мобильн», Flutter, React Native, Kotlin Multiplatform (KMP); короткие токены `ios`, `swift`, `kmp` ищутся как отдельные слова, чтобы не ловить подстроки;
3. в названии нет признаков другой профессии: тестирование/QA/SDET, аналитика, дизайн, менеджмент, преподавание, а также «автомобил» (иначе «Программист 1С автомобильный бизнес» попадает в выборку по подстроке «мобильн»).

Навыки вакансии нормализуются одинаково в обоих способах: `trim`, нижний регистр, удаление дублей и пустых строк. Условие написано отдельно на Python (для MapReduce) и на SQL (для DuckDB):

```
# hhlake/variant.py
MOBILE_TITLE = re.compile(
    r"android|mobile|мобильн|flutter|react[ -]?native|kotlin multiplatform"
    r"|(?<![a-z])(?:ios|swift|kmp)(?![a-z])", re.IGNORECASE)
EXCLUDED_TITLE = re.compile(
    r"тестир|аналитик|analyst|дизайн|design|автомобил|менеджер|manager|руководитель проект"
    r"|преподава|sdet|in test|(?<![a-z])qa(?![a-z])", re.IGNORECASE)

def is_target(vacancy: dict) -> bool:
    title = vacancy.get("name") or ""
    roles = {role["id"] for role in vacancy.get("professional_roles") or []}
    return (MOBILE_TITLE.search(title) is not None and EXCLUDED_TITLE.search(title) is None
            and not roles.isdisjoint(DEVELOPER_ROLES))
```

```
-- sql/target.sql
WHERE regexp_matches(
        name,
        'android|mobile|мобильн|flutter|react[ -]?native|kotlin multiplatform'
        || '|(^|[^a-z])(ios|swift|kmp)([^a-z]|$)',
        'i')
  AND NOT regexp_matches(
        name,
        'тестир|аналитик|analyst|дизайн|design|автомобил|менеджер|manager'
        || '|руководитель проект|преподава|sdet|in test|(^|[^a-z])qa([^a-z]|$)',
```

```
      'i')
  AND list_has_any(list_transform(professional_roles, lambda r: r.id), ['96', '104']);
```

Под условие попадают **142 вакансии**: от «IOS разработчик» и «Android Developer» до «Team Lead Mobile» и «Flutter-разработчик». У 40 из них список `key_skills` пуст.

Подсчёт навыков: собственный MapReduce

Движок `hhlake/mapreduce.py` работает напрямую с S3 и не скачивает объект на диск.

- **Splits.** Размер объекта берётся через `HEAD`, объект делится на диапазоны байт $[start, end)$ по `--chunk-mb` МиБ (по умолчанию 8 МиБ, 7 сплитов).
- **Чтение чанка.** Каждая map-задача делает `GET` с заголовком `Range`. Граница чанка может разрезать строку, поэтому применяется правило Hadoop LineRecordReader: строка принадлежит тому сплиту, в котором лежит её первый байт. Задача читает с байта $start - 1$ и отбрасывает всё до первого перевода строки (это хвост чужой строки), а последнюю строку дочитывает за пределами end дополнительными range-запросами.
- **Map.** Строка разбирается, проверяется условие варианта, для отсортированного списка навыков эмитируются все сочетания размера 1, 2 и 3 (`itertools.combinations`), а также служебные ключи для числа вакансий.
- **Combiner и партиционирование.** Внутри маппера пары (*ключ*, 1) суммируются в `Counter` по партициям. Номер партиции равен CRC32 ключа по модулю числа редьюсеров. Это стабильный хэш, в отличие от встроенного `hash()`, который рандомизирован в каждом процессе.
- **Shuffle и Reduce.** Драйвер собирает r-ю партицию всех мапперов и отдаёт её r-му редьюсеру, который суммирует значения.
- **Отказоустойчивость.** Драйвер следит за futures. Если map-задача упала, её частичный результат отбрасывается, и задача перезапускается (до трёх попыток). Флаг `--fail-split N` внедряет падение в середине чанка N на первой попытке.

Мапперы и редьюсеры по умолчанию — 4 процесса (`ProcessPoolExecutor`). Результаты трёх размеров стека пишутся в префикс `s3://hh-lake/results/mapreduce/` (файлы `skills.csv`, `pairs.csv`, `triples.csv`), сводка — в `summary.json`.

```
def iter_lines_buffered(bucket, key, size, split, stats):
    lo = split.start - 1 if split.start else 0
    buf = storage.read_range(bucket, key, lo, split.end)    # GET Range: bytes=lo-(end-1)
    if split.start:
        newline = buf.find(b"\n")
        if newline < 0:
            return                                          # no line starts in this split
        buf = buf[newline + 1:]                             # tail of the previous line
    if not buf:
        return
    pos = split.end
    while not buf.endswith(b"\n") and pos < size:           # finish the last line
        more = storage.read_range(bucket, key, pos, min(pos + TAIL_STEP, size))
        ...
```

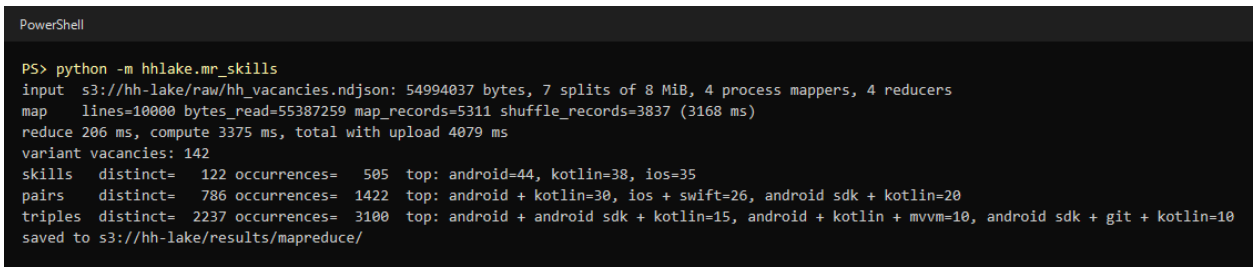


Рис. 4: MapReduce: 7 сплитов по 8 МиБ, 10 000 строк, 142 вакансии варианта; навыки, пары и тройки сохранены в хранилище.

Тест отказа мапперов (`scripts/failover_test.ps1`). Задачи 2 и 5 падают на середине своих чанков (после 748 и 783 строк). Драйвер отбрасывает их частичный вывод и перезапускает задачи. Итог — те же 10 000 строк без двойного счёта, а результат совпадает с DuckDB построчно.

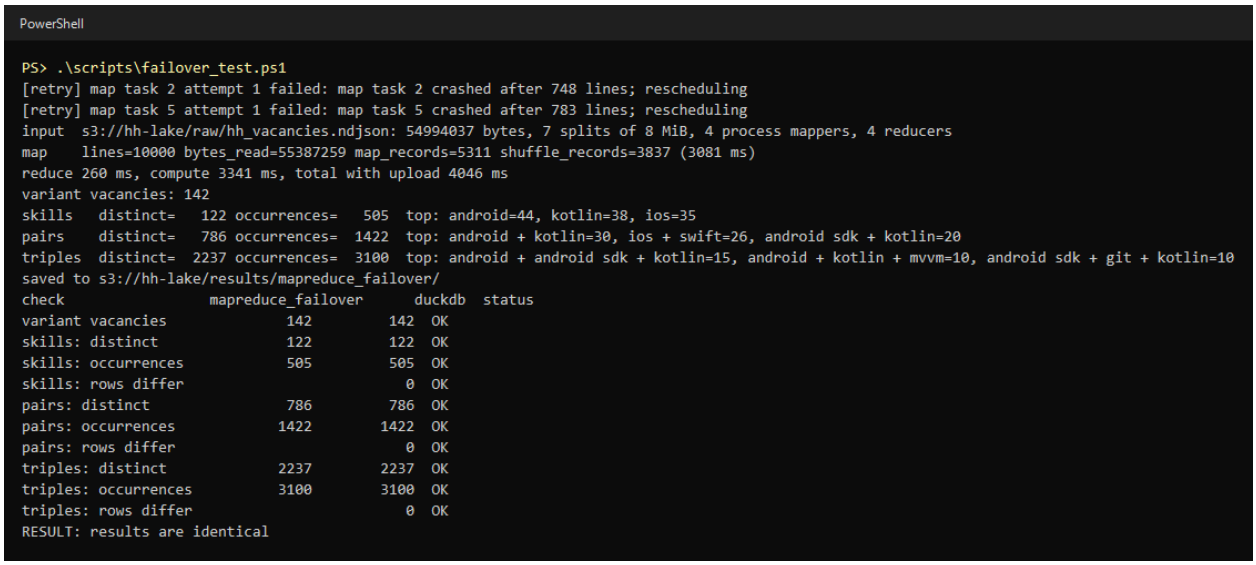


Рис. 5: Отказ двух мапперов: задачи перезапущены, результат идентичен эталону.

Независимость от размера чанка (`scripts/chunk_test.ps1`). При чанках 0,5, 2 и 32 МиБ границы режут строки в разных местах, а результат каждый раз совпадает с DuckDB. Это проверяет корректность обработки границ. Число записей shuffle растёт с числом сплитов (3410 при 2 сплитах, 4442 при 105), потому что combiner агрегирует только внутри своего чанка.

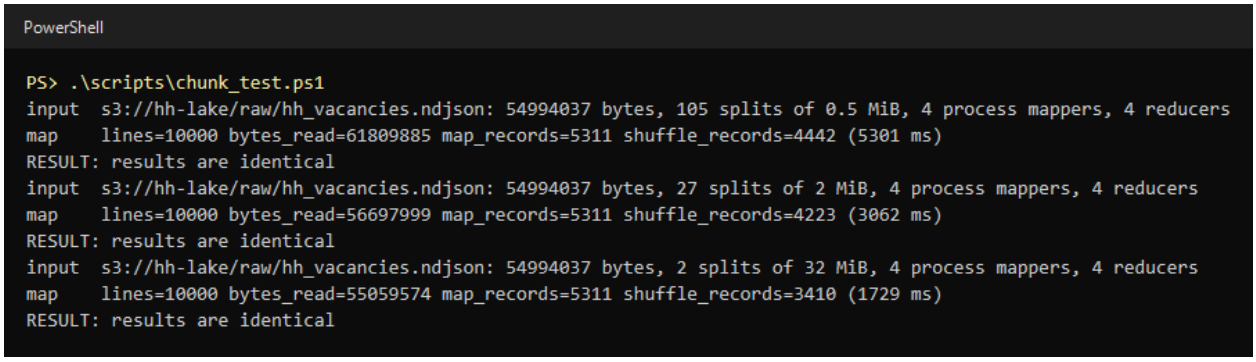


Рис. 6: Разные размеры чанка дают идентичный результат.

Подсчёт навыков: DuckDB

DuckDB читает тот же объект через расширение `httpfs` (секрет S3 с эндпоинтом RustFS, path-style). В `sql/vacancies.sql` схема JSON задана явно, чтобы не зависеть от автоопределения типов. Таблица `target` строится условием варианта, приведённым выше. Пары и тройки получаются самосоединением таблицы «вакансия–навык» со строгим порядком навыков, что даёт каждое сочетание ровно один раз:

```
CREATE OR REPLACE TABLE vacancy_skill AS SELECT vacancy_id, unnest(skills) AS skill FROM target;

CREATE OR REPLACE TABLE stack_triples AS
SELECT a.skill AS skill_1, b.skill AS skill_2, c.skill AS skill_3, count(*) AS vacancies
FROM vacancy_skill a
JOIN vacancy_skill b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
JOIN vacancy_skill c ON b.vacancy_id = c.vacancy_id AND b.skill < c.skill
GROUP BY ALL ORDER BY vacancies DESC, skill_1, skill_2, skill_3;
```

```
PowerShell

PS> python -m hhlake.duckdb_skills
duckdb v2.0.0, source raw, compute 2232 ms, total 3108 ms
variant vacancies: 142
skills   distinct=   122 occurrences=   505  top: android=44, kotlin=38, ios=35
pairs    distinct=   786 occurrences=  1422  top: android + kotlin=30, ios + swift=26, android sdk + kotlin=20
triples  distinct=  2237 occurrences=  3100  top: android + android sdk + kotlin=15, android + kotlin + mvvm=10, android sdk + git + kotlin=10
saved to s3://hh-lake/results/duckdb/
```

Рис. 7: DuckDB: те же 142 вакансии, результаты сохранены в `results/duckdb/`.

Верификация результатов

`hhlake/verify.py` скачивает из хранилища CSV обоих способов и сравнивает их как словари «сочетание → число вакансий»: число различных ключей, сумму вхождений и число расходящихся строк.

```
PowerShell

PS> python -m hhlake.verify mapreduce duckdb
check                    mapreduce      duckdb  status
variant vacancies              142         142  OK
skills: distinct               122         122  OK
skills: occurrences            505         505  OK
skills: rows differ                          0  OK
pairs: distinct                786         786  OK
pairs: occurrences            1422        1422  OK
pairs: rows differ                           0  OK
triples: distinct              2237        2237  OK
triples: occurrences           3100        3100  OK
triples: rows differ                         0  OK
RESULT: results are identical
```

Рис. 8: Контрольные числа MapReduce и DuckDB совпадают, расходящихся строк 0.

Таблица 1: Контрольные числа (оба способа)

	Навыки	Пары	Тройки
Различных сочетаний	122	786	2 237
Вхождений (сумма по вакансиям)	505	1 422	3 100
Расходящихся строк	0	0	0

Сверка уже окупилась в ходе работы. После того как в Python-фильтр добавили

исключение «SDET / in test», SQL-версию забыли обновить, и `verify` сразу показал `DIFF`: 142 против 144 вакансий, у `java` 14 против 16, лишний навык `qa automation`. После правки SQL результаты совпали.

Результаты: навыки, пары и тройки

Таблица 2: Топ-10 навыков, пар и троек (число вакансий)

Навык	N	Пара	N	Тройка	N
android	44	android + kotlin	30	android + android sdk + kotlin	15
kotlin	38	ios + swift	26	android + kotlin + mvvm	10
ios	35	android sdk + kotlin	20	android sdk + git + kotlin	10
swift	35	android + android sdk	18	android + android sdk + java	9
git	29	git + kotlin	14	android + git + kotlin	9
android sdk	23	android sdk + java	12	android sdk + java + kotlin	9
mvvm	16	git + swift	12	android + kotlin + mvp	8
java	14	kotlin + mvvm	11	android + mvp + mvvm	8
react native	13	android + git	10	kotlin + mvp + mvvm	8
mvp	10	android + ios	10	android + android sdk + git	7

Рынок мобильной разработки делится на два нативных лагеря: Android (Kotlin, Android SDK, а в legacy-проектах Java) и iOS (Swift). Пары `android + kotlin` (30) и `ios + swift` (26) — это ядро стека платформы. Из архитектурных паттернов чаще всего требуют MVVM и MVP. Кроссплатформа (React Native, Flutter) заметно реже нативной разработки.

Рост числа пар и троек. Вакансия с k навыками даёт $\binom{k}{1} = k$ навыков, $\binom{k}{2}$ пар и $\binom{k}{3}$ троек. Число сочетаний растёт полиномиально: при $k = 4$ это 6 пар и 4 тройки, при $k = 14$ — уже 91 пара и 364 тройки. `hhlake/combinatorics.py` берёт распределение числа навыков по вакансиям из сводки MapReduce и проверяет, что $\sum \binom{k}{n}$ точно равно числу вхождений.

```
PowerShell

PS> python -m hhlake.combinatorics
k skills  vacancies  C(k,1)  C(k,2)  C(k,3)   skills    pairs  triples
      0         40       0       0       0        0        0        0
      1          8       1       0       0        8        0        0
      2         12       2       1       0       24       12        0
      3         12       3       3       1       36       36       12
      4         24       4       6       4       96      144       96
      5         10       5      10      10       50      100      100
      6         12       6      15      20       72      180      240
      7          7       7      21      35       49      147      245
      8          5       8      28      56       40      140      280
      9          5       9      36      84       45      180      420
     10          1      10      45     120       10       45      120
     11          3      11      55     165       33      165      495
     14          3      14      91     364       42      273     1092
  total        142                            505     1422     3100
skills   sum C(k,1) =   505  MapReduce occurrences =   505  distinct =   122  avg vacancies per distinct = 4.14
pairs    sum C(k,2) =  1422  MapReduce occurrences =  1422  distinct =   786  avg vacancies per distinct = 1.81
triples  sum C(k,3) =  3100  MapReduce occurrences =  3100  distinct =  2237  avg vacancies per distinct = 1.39
```

Рис. 9: Сумма $\binom{k}{n}$ по вакансиям совпадает с числом вхождений: 505, 1 422, 3 100.

Из этой таблицы следуют три наблюдения.

- При переходе от навыков к тройкам число вхождений растёт в 6,1 раза (505 → 3 100), а число различных сочетаний — в 18 раз (122 → 2 237).
- Тройки дают в основном «длинные» вакансии. Всего три вакансии с 14 навыками порождают 1 092 тройки, это 35% всех вхождений троек. Больше половины вакансий с навыками (56 из 102, у них $k \leq 4$) дают лишь 108 троек.
- Чем больше размер сочетания, тем сильнее разрежен результат. Среднее число вакан-

сий на одно сочетание падает с 4,14 (навыки) до 1,81 (пары) и 1,39 (тройки). 1 587 из 2 237 троек (71%) встречаются ровно в одной вакансии. Поэтому для анализа зарплат по тройкам нужен порог минимальной поддержки.

Влияние навыков на заработную плату

Методика. Зарплата приводится к месячной сумме «на руки» в рублях: берётся середина вилки или единственная указанная граница, `gross` умножается на 0,87, USD и EUR пересчитываются по фиксированным курсам середины 2024 года (90 и 98 руб.). Зарплата указана у 34 из 142 вакансий (32 в рублях и 2 в долларах), у 28 из них есть навыки. Медиана — 160 000 руб., межквартильный размах — 101 250–200 075 руб..

Прибавка считается так, чтобы было видно, что добавляет дополнительный навык:

- для пары базой служит **более частый** навык пары (частота по всем 142 вакансиям), прибавка = медиана пары − медиана базового навыка;
- для тройки базой служит **самая частая** из трёх входящих в неё пар, прибавка = медиана тройки − медиана базовой пары;
- учитываются сочетания, встретившиеся минимум в 3 вакансиях с зарплатой (34 пары и 26 троек).

Уровень зарплаты и набор навыков. Вакансии разделены по медиане на «высокие» (18, медиана 200 050 руб.) и «низкие» (16, медиана 98 503 руб.). Количество навыков почти не отличается: 4,6 против 4,9. Решает не длина списка навыков, а его состав.

- Только в высоких вакансиях встречаются английский язык (22%), Objective-C (17%) и REST (17%).
- В низких вакансиях заметно чаще встречаются Git (63% против 22%), Flutter, Dart и SOLID (по 0% в высоких), REST API и React Native.

PowerShell

```
PS> python -m hhlake.salary | Select-Object -First 44
== salary overview, RUB net per month
```

vacancies int64	with_salary int64	with_salary_and_skills int64	q1 double	median double	q3 double
142	34	28	101250.0	160000.0	200075.0

== salary bands: at/above vs below the median

band varchar	vacancies int64	median_salary double	avg_skills double	without_skills int64
high	18	200050.0	4.6	4
low	16	98503.0	4.9	2

== skills in high vs low salary band (>= 3 salaried vacancies)

skill varchar	high int64	low int64	high_pct double	low_pct double
английский язык	4	0	22.0	0.0
objective-c	3	0	17.0	0.0
rest	3	0	17.0	0.0
android sdk	5	3	28.0	19.0
ios	6	4	33.0	25.0
json	2	1	11.0	6.0
react	2	1	11.0	6.0
sql	2	1	11.0	6.0
mvvm	2	2	11.0	13.0
swift	4	4	22.0	25.0
android	7	7	39.0	44.0
javascript	1	2	6.0	13.0
java	2	3	11.0	19.0
kotlin	4	5	22.0	31.0
react native	2	4	11.0	25.0
dart	0	3	0.0	19.0
rest api	1	4	6.0	25.0
solid	0	3	0.0	19.0
flutter	0	4	0.0	25.0
git	4	10	22.0	63.0

20 rows 5 columns

Рис. 10: Обзор зарплат и сравнение навыков в высоких и низких вакансиях.

PowerShell

```
PS> python -m hhlake.salary | Select-Object -Skip 45 -First 29
== pairs: top 8 raising the median vs the more frequent skill
```

base varchar	base_freq int64	base_n int64	base_median double	added varchar	pair_n int64	pair_median double	uplift double	uplift_pct double
swift	35	8	178300.0	objective-c	3	225000.0	46700.0	26.2
react native	13	6	145000.0	react	3	175000.0	30000.0	20.7
ios	35	10	200000.0	objective-c	3	225000.0	25000.0	12.5
git	29	14	137500.0	mvvm	4	158300.0	20800.0	15.1
android	44	14	147500.0	android sdk	7	160000.0	12500.0	8.5
android	44	14	147500.0	rest	3	160000.0	12500.0	8.5
android	44	14	147500.0	sql	3	160000.0	12500.0	8.5
android	44	14	147500.0	английский язык	3	160000.0	12500.0	8.5

```
== pairs: top 8 lowering the median vs the more frequent skill
```

base varchar	base_freq int64	base_n int64	base_median double	added varchar	pair_n int64	pair_median double	uplift double	uplift_pct double
ios	35	10	200000.0	flutter	3	105000.0	-95000.0	-47.5
android sdk	23	8	160000.0	solid	3	78300.0	-81700.0	-51.1
kotlin	38	9	156600.0	java	3	78300.0	-78300.0	-50.0
kotlin	38	9	156600.0	solid	3	78300.0	-78300.0	-50.0
android	44	14	147500.0	solid	3	78300.0	-69200.0	-46.9
git	29	14	137500.0	java	3	78300.0	-59200.0	-43.1
git	29	14	137500.0	solid	3	78300.0	-59200.0	-43.1
java	14	5	130500.0	solid	3	78300.0	-52200.0	-40.0

Рис. 11: Пары, сильнее всего повышающие и понижающие медиану относительно более частого навыка.

PowerShell

```
PS> python -m hhlake.salary | Select-Object -Skip 75 -First 26
== triples: top 6 raising the median vs the most frequent pair
```

base varchar	base_freq int64	base_n int64	base_median double	added varchar	triple_n int64	triple_median double	uplift double	uplift_pct double
git + kotlin	14	7	130500.0	mvvm	3	156600.0	26100.0	20.0
ios + swift	26	7	200000.0	objective-c	3	225000.0	25000.0	12.5
ios + swift	26	7	200000.0	git	4	205538.0	5538.0	2.8
android + android sdk	18	7	160000.0	английский язык	3	160000.0	0.0	0.0
android + kotlin	30	7	130500.0	android sdk	5	130500.0	0.0	0.0
git + java	6	3	78300.0	solid	3	78300.0	0.0	0.0

```
== triples: top 6 lowering the median vs the most frequent pair
```

base varchar	base_freq int64	base_n int64	base_median double	added varchar	triple_n int64	triple_median double	uplift double	uplift_pct double
ios + swift	26	7	200000.0	android	3	105000.0	-95000.0	-47.5
android + android sdk	18	7	160000.0	solid	3	78300.0	-81700.0	-51.1
android sdk + kotlin	20	6	145250.0	java	3	78300.0	-66950.0	-46.1
android sdk + kotlin	20	6	145250.0	solid	3	78300.0	-66950.0	-46.1
android + android sdk	18	7	160000.0	git	4	104400.0	-55600.0	-34.8
android + git	10	7	130500.0	java	3	78300.0	-52200.0	-40.0

Рис. 12: Тройки, сильнее всего повышающие и понижающие медиану относительно самой частой пары.

Таблица 3: Пары и тройки, сильнее всего меняющие медианную зарплату (руб. на руки)

База (медиана, n)	+ навык	n	Медиана	Прибавка	%
Повышают					
swift (178 300, 8)	objective-c	3	225 000	+46 700	+26,2
react native (145 000, 6)	react	3	175 000	+30 000	+20,7
git + kotlin (130 500, 7)	mvvm	3	156 600	+26 100	+20,0
ios (200 000, 10)	objective-c	3	225 000	+25 000	+12,5
ios + swift (200 000, 7)	objective-c	3	225 000	+25 000	+12,5
git (137 500, 14)	mvvm	4	158 300	+20 800	+15,1
android (147 500, 14)	android sdk	7	160 000	+12 500	+8,5
Понижают					
ios (200 000, 10)	flutter	3	105 000	−95 000	−47,5
ios + swift (200 000, 7)	android	3	105 000	−95 000	−47,5
android sdk (160 000, 8)	solid	3	78 300	−81 700	−51,1
android + android sdk (160 000, 7)	solid	3	78 300	−81 700	−51,1
kotlin (156 600, 9)	java	3	78 300	−78 300	−50,0
android sdk + kotlin (145 250, 6)	java	3	78 300	−66 950	−46,1
android + android sdk (160 000, 7)	git	4	104 400	−55 600	−34,8

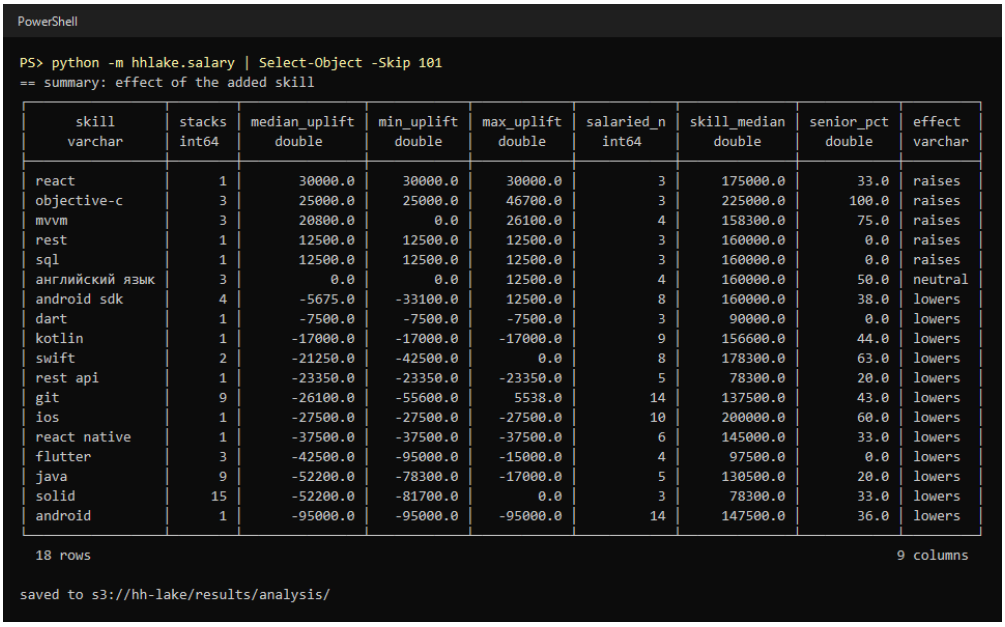


Рис. 13: Итог по дополнительному навыку: медианная прибавка по всем парам и тройкам, где он добавляется, и доля вакансий с опытом от 3 лет.

Таблица 4: Итоговая таблица: навыки, повышающие и понижающие зарплату

Навык	Медианная прибавка	Интерпретация
Повышают		
Objective-C	+25 000 (3 сочет.)	Legacy-кодовая база iOS. 100% таких вакансий требуют опыт от 3 лет, это маркер senior-позиции, а не языка как такового.
MVVM	+20 800 (3)	Требование к архитектуре приложения. 75% вакансий — senior.
React (к React Native)	+30 000 (1)	Разработчик закрывает и веб, и мобильный клиент: шире зона ответственности.
REST, SQL, английский	+12 500 / 0	Работа с бэкендом и международными командами, умеренная надбавка к Android.
Понижают		
Flutter, Dart	−42 500 (3)	Кроссплатформа во всех случаях с зарплатой — региональные вакансии (Краснодар 45 000, Киров 90 000, Тюмень 105 000, СПб 135 000), 0% senior. Один кроссплатформенный разработчик заменяет двух нативных, работодатели этого сегмента экономят. Тот же эффект даёт «+android» к `ios + swift`.
SOLID, Java	−52 200 (15 и 9)	Эффект создают две одинаковые вакансии одного работодателя в Ростове-на-Дону и Краснодаре (одни и те же 9 навыков, 78 300 руб.) и ярославская вакансия. Java в Android-вакансии — признак legacy-стека и регионального работодателя. Понижение связано с географией и дублями, а не с самим навыком.
Git	−26 100 (9)	Базовый инструмент, который явно перечисляют в вакансиях уровня junior/middle (63% низких вакансий против 22% высоких). В senior-вакансиях очевидные инструменты не пишут.

Ограничения. 34 вакансии с зарплатой — малая выборка. При поддержке 3 вакансии медиана сочетания определяется одной-двумя вакансиями. Поэтому выводы следует считать гипотезами: они указывают на сегменты (senior iOS, кроссплатформа, региональный Android), а не на точную цену навыка.

Региональный анализ

Выбраны Москва, Санкт-Петербург и Краснодар — регионы, где есть мобильные вакансии с зарплатой, без чего нельзя посчитать медиану (у Казани и Новосибирска таких вакансий нет). В качестве контекста посчитаны показатели всего IT-рынка региона (все 10 000 вакансий датасета).

PowerShell

```
PS> python -m hhlake.regions | Select-Object -First 18
== region metrics: salaries
```

area varchar	it_vacancies int64	vacancies int64	share_of_it_pct double	with_salary int64	q1 double	median double	q3 double	it_median double	premium_vs_it_pct double
Москва	3795	57	1.5	12	193750.0	208800.0	228619.0	160000.0	30.0
Санкт-Петербург	1278	18	1.4	2	138750.0	142500.0	146250.0	108750.0	31.0
Краснодар	222	5	2.3	3	61650.0	78300.0	151650.0	73500.0	7.0

```
== region metrics: vacancy profile
```

area varchar	avg_skills double	no_skills_pct double	remote_pct double	senior_pct double	android int64	ios int64	cross_platform int64
Москва	4.3	39.0	47.0	60.0	25	24	10
Санкт-Петербург	5.0	28.0	22.0	61.0	6	7	4
Краснодар	6.8	0.0	40.0	60.0	2	1	2

Рис. 14: Показатели регионов: зарплаты и профиль вакансий.

Таблица 5: Ключевые показатели регионов (вариант 9)

Показатель	Москва	Санкт-Петербург	Краснодар
IT-вакансий в регионе	3 795	1 278	222
Мобильных вакансий (доля от IT)	57 (1,5%)	18 (1,4%)	5 (2,3%)
С зарплатой	12	2	3
Медиана, руб. на руки	208 800	142 500	78 300
Квартили Q1–Q3, руб.	193 750–228 619	138 750–146 250	61 650–151 650
Медиана всего IT, руб.	160 000	108 750	73 500
Премия мобильных к IT	+30%	+31%	+7%
Навыков на вакансию	4,3	5,0	6,8
Вакансий без навыков	39%	28%	0%
Удалённая работа	47%	22%	40%
Опыт от 3 лет	60%	61%	60%
Android / iOS / кроссплатформа	25 / 24 / 10	6 / 7 / 4	2 / 1 / 2

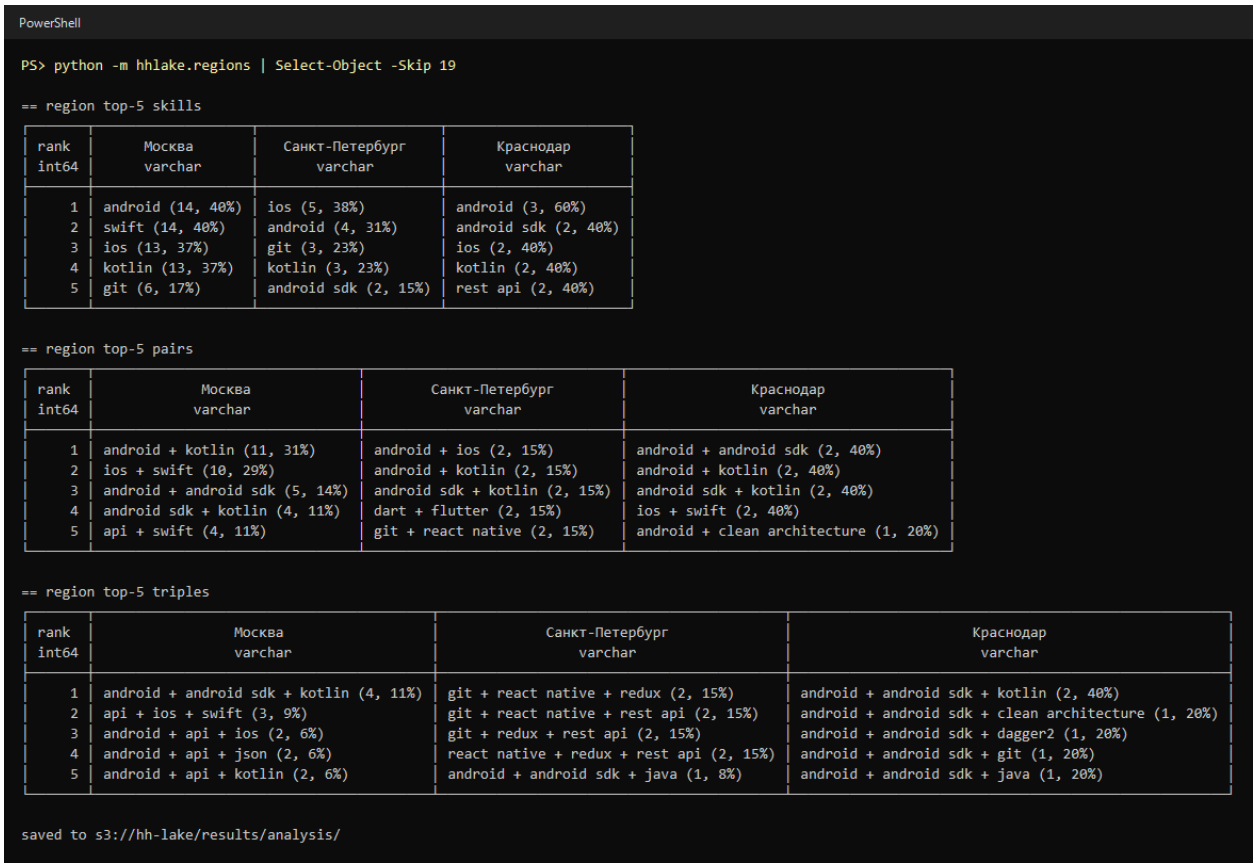


Рис. 15: Топ-5 навыков, пар и троек по регионам (число вакансий и доля от вакансий региона с навыками).

Таблица 6: Самые востребованные навыки, пары и тройки по регионам

	Москва	Санкт-Петербург	Краснодар
Навыки	android 40%, swift 40%, ios 37%, kotlin 37%	ios 38%, android 31%, git 23%, kotlin 23%	android 60%, android sdk 40%, ios 40%, kotlin 40%
Пары	android + kotlin 31%, ios + swift 29%, android + android sdk 14%	android + ios 15%, android + kotlin 15%, dart + flutter 15%	android + android sdk 40%, android + kotlin 40%, ios + swift 40%
Тройки	android + android sdk + kotlin 11%, api + ios + swift 9%	git + react native + redux 15%, react native + redux + rest api 15%	android + android sdk + kotlin 40%, android + android sdk + clean architecture 20%

Наблюдения и инсайты.

- **Москва концентрирует рынок.** В ней 57 из 142 вакансий варианта (40%) и 12 из 34 зарплат. Android и iOS в паритете (25 и 24 вакансии), топ пар и троек — чистый нативный стек: `android + kotlin`, `ios + swift`.
- **Зарплатный разрыв задаёт география, а не опыт.** Доля вакансий с опытом от 3 лет одинакова во всех трёх регионах (60–61%), а медиана в Москве в 1,47 раза выше петербургской и в 2,7 раза выше краснодарской.
- **Мобильная разработка — премиальный сегмент IT в столицах.** Медиана мобильных вакансий выше медианы всего IT-рынка на 30% в Москве и на 31% в Петербурге, но лишь на 7% в Краснодаре. В регионе мобильный разработчик оплачивается почти как средний IT-специалист.
- **Петербург смещён в кроссплатформу на JavaScript.** Весь топ троек занят стеком React Native + Redux + REST API, среди пар есть `dart + flutter`. Доля кроссплатформенных вакансий — 22% против 18% в Москве.
- **Москва нанимает удалённо.** 47% московских вакансий удалённые против 22% в Петербурге, то есть московские работодатели конкурируют за разработчиков из регионов. Для разработчика из Краснодара удалённая московская вакансия означает медиану 208 800 против 78 300 руб. локально.
- **Стиль описания вакансий различается.** В 39% московских вакансий `key_skills` пусты: крупные компании пишут требования в текст описания. Региональные работодатели перечисляют длинные списки инструментов (6,8 навыка на вакансию в Краснодаре, включая Clean Architecture и Dagger2).

Ограничения. Медиана Петербурга посчитана по 2 вакансиям, Краснодара — по 3, причём одна из них — вакансия работодателя, разместившего такую же в Ростове-на-Дону. Выводы по этим регионам качественные.

Профилирование MapReduce и DuckDB

Методика. `hhlake/profiler.py` запускает каждую конфигурацию отдельным процессом 5 раз подряд. Каждые 5 мс он суммирует RSS (working set) процесса и всех его потомков, поэтому в пик памяти входят и рабочие процессы пула MapReduce. Фиксируются две метрики времени:

- **wall** — полное время процесса: старт интерпретатора, импорты, расчёт, запись результатов в S3;
- **compute** — время самого расчёта, которое процесс сообщает о себе: от начала чтения из S3 до получения всех частот, без старта и выгрузки.

Разница wall − compute (1,6–1,9 с) — постоянные накладные расходы: старт Python, импорт boto3 и DuckDB, четыре PUT в RustFS. Сравнивать подходы корректнее по compute. Все замеры — `scripts/profile.ps1`, сырые данные — `results/profile.csv`.

PowerShell

```
PS> Import-Csv results\profile.csv | Format-Table name, run, seconds, compute_seconds, peak_mib

name             run seconds compute_seconds peak_mib
----             --- ------- --------------- --------
mr_v1_baseline   1   4.155   2.562           327.3
mr_v1_baseline   2   3.728   1.947           327.6
mr_v1_baseline   3   3.468   1.824           329.1
mr_v1_baseline   4   3.474   2.013           326.8
mr_v1_baseline   5   3.687   2.147           329.3
mr_v2_streaming  1   3.402   1.732           283.8
mr_v2_streaming  2   3.634   1.937           285.2
mr_v2_streaming  3   3.237   1.728           285.0
mr_v2_streaming  4   3.767   1.919           285.2
mr_v2_streaming  5   3.266   1.710           284.3
mr_v3_threads    1   2.698   1.114           63.5
mr_v3_threads    2   2.680   1.170           63.6
mr_v3_threads    3   3.027   1.459           63.8
mr_v3_threads    4   2.783   1.109           63.1
mr_v3_threads    5   2.741   1.193           63.6
mr_v4_prefilter  1   2.682   0.984           62.2
mr_v4_prefilter  2   2.755   1.070           63.1
mr_v4_prefilter  3   2.556   1.024           63.0
mr_v4_prefilter  4   3.040   1.362           62.7
mr_v4_prefilter  5   3.000   1.364           63.1
mr_v5_curated    1   2.073   0.431           60.3
mr_v5_curated    2   2.032   0.419           60.3
mr_v5_curated    3   1.918   0.426           60.3
mr_v5_curated    4   2.159   0.430           59.9
mr_v5_curated    5   2.264   0.415           60.1
duckdb_v1_raw    1   2.738   0.931           194.3
duckdb_v1_raw    2   2.801   0.979           185.0
duckdb_v1_raw    3   2.961   0.980           191.6
duckdb_v1_raw    4   2.798   0.936           196.7
duckdb_v1_raw    5   2.740   0.913           170.0
duckdb_v2_parquet 1  1.994   0.233           96.3
duckdb_v2_parquet 2  2.222   0.266           96.5
duckdb_v2_parquet 3  2.128   0.228           97.2
duckdb_v2_parquet 4  2.136   0.243           96.1
duckdb_v2_parquet 5  1.995   0.217           97.6
```

Рис. 16: Все 35 запусков (7 конфигураций по 5 попыток): wall time, compute time, пик памяти.

PowerShell

```
PS> python -m hhlake.profile_report
configuration       runs  wall s     sd  compute s     sd    min    max  peak MiB     min     max
mr_v1_baseline         5   3.702  0.280      2.099  0.284  1.824  2.562     328.0   326.8   329.3
mr_v2_streaming        5   3.461  0.232      1.805  0.113  1.710  1.937     284.7   283.8   285.2
mr_v3_threads          5   2.786  0.141      1.209  0.144  1.109  1.459      63.5    63.1    63.8
mr_v4_prefilter        5   2.807  0.208      1.161  0.187  0.984  1.364      62.8    62.2    63.1
mr_v5_curated          5   2.089  0.131      0.424  0.007  0.415  0.431      60.2    59.9    60.3
duckdb_v1_raw          5   2.808  0.091      0.948  0.030  0.913  0.980     187.5   170.0   196.7
duckdb_v2_parquet      5   2.095  0.099      0.237  0.019  0.217  0.266      96.7    96.1    97.6
```

Рис. 17: Сводка профилирования: среднее, стандартное отклонение, минимум и максимум.

Сравнение подходов на одинаковых данных (сырой NDJSON, 55 МБ):

Подход	wall, с	compute, с (sd)	compute min–max	Пик памяти, МиБ
MapReduce (4 процесса, 8 МиБ)	3,70	2,10 (0,28)	1,82–2,56	328
DuckDB (raw NDJSON)	2,81	0,95 (0,03)	0,91–0,98	188

DuckDB быстрее в 2,2 раза по compute и потребляет в 1,7 раза меньше памяти. Причины такие.

- **Чтение и разбор.** JSON-ридер DuckDB — векторизованный многопоточный C++: 16 потоков, параллельные range-запросы к S3 через httpfs, материализуются только 8 объявленных полей. MapReduce разбирает каждую строку целиком, включая описание (основную массу байт), стандартным `json` под GIL.
- **Процессы.** На Windows пул запускает процессы через `spawn`, и каждый из 4 воркеров

заново поднимает интерпретатор и импортирует boto3 (около 0,4 с и 55–65 МиБ на процесс). Из 328 МиБ около 260 приходится на воркеры.
- **Буферы.** v1 держит весь чанк (8 МиБ) в памяти и копирует его при разбиении на строки.
- **Стабильность.** Разброс у MapReduce выше (sd 0,28 с против 0,03 с): он чувствителен к планированию процессов и к задержкам отдельных GET-запросов.

Задача при этом I/O-bound: один GET всего объекта из RustFS идёт 1,4 с (около 40 МБ/с через EC и 9p-тома WSL2), а разбор всего JSON занимает 0,3 с.

Бонус: оптимизация расчётов

Оптимизации вводились по одной, и каждая версия профилировалась 5 раз (таблица выше, строки `mr_v1`…`mr_v5`, `duckdb_v1/v2`). Результат каждой версии сохранён в `results/profile/<версия>/` и сверен с эталоном (`scripts/verify_profile.ps1`).

Таблица 7: MapReduce и DuckDB: «до» и «после» (среднее по 5 запускам)

Версия	wall, с	compute, с	Пик, МиБ	Читается из S3
mr_v1: процессы, буфер на чанк 8 МиБ	3,70	2,10	328	55,39 МБ
mr_v2: + потоковое чтение чанка	3,46	1,81	285	55,09 МБ
mr_v3: + потоки вместо процессов, 8 мапперов, 4 МиБ	2,79	1,21	63,5	55,2 МБ
mr_v4: + префильтр строки по id роли	2,81	1,16	62,8	55,2 МБ
mr_v5: + curated NDJSON (только нужные поля)	2,09	**0,42**	**60,2**	5,2 МБ
duckdb_v1: raw NDJSON	2,81	0,95	188	55,0 МБ
duckdb_v2: curated Parquet (zstd)	2,10	**0,24**	**96,7**	0,39 МБ

Что дала каждая оптимизация.

1. **Потоковое чтение** (`iter_lines_streaming`). Чанк читается из тела ответа кусками по 64 КиБ, без буфера на весь чанк. Хвост последней строки дочитывается шагами 16 КиБ с удвоением, а не фиксированными 64 КиБ. Пик памяти снизился на 43 МиБ (по ≈10 МиБ на воркер), compute — на 14%. Лишнее чтение хвостов сократилось с 393 КБ до 98 КБ (на мелких чанках 0,5 МиБ в v1 читалось 61,8 МБ вместо 55).
2. **Потоки вместо процессов.** Задача I/O-bound: сетевое чтение отпускает GIL, а на разбор JSON приходится меньше пятой части времени. Поэтому `ThreadPoolExecutor` не проигрывает процессам в параллелизме, зато исчезают spawn и 4 копии интерпретатора с boto3. Восемь потоков по 4 МиБ (14 сплитов) дают больше параллельных GET. Память упала с 285 до 63 МиБ (в 4,5 раза), compute — с 1,81 до 1,21 с.
3. **Префильтр.** JSON строки не разбирается, если в ней нет токена `"96"` или `"104"`. Это необходимое условие роли разработчика: любая сериализация JSON содержит значение id в кавычках. Выигрыш (1,21 → 1,16 с) меньше стандартного отклонения (0,14–0,19 с) и статистически незначим. Узкое место не CPU, а ввод-вывод, поэтому ускорение разбора мало что даёт. Оптимизация оставлена, но эффект честно признан нулевым.
4. **Curated-слой** (`hhlake/curate.py`, `sql/curate.sql`). Разовый ETL из сырого слоя за 1,9 с оставляет только 8 используемых полей и выбрасывает описания. Получаются `curated/vacancies.parquet` (387 КБ, 0,7% сырого объекта, колоночный zstd) и `curated/vacancies.ndjson` (5,1 МБ, 9,3%). MapReduce ускорился с 1,16 до 0,42 с (разброс упал до 0,007 с), DuckDB — с 0,95 до 0,24 с, память DuckDB уменьшилась вдвое. Затраты на ETL окупаются уже на втором-третьем запуске анализа: в работе их больше десятка — навыки, зарплаты, регионы, профилирование.

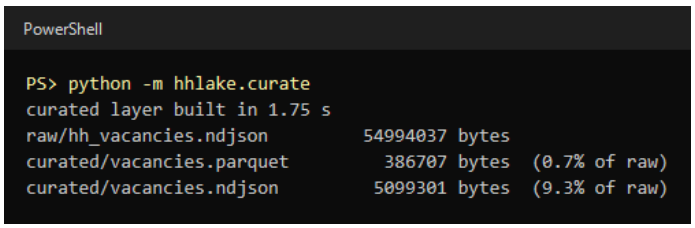


Рис. 18: Curated-слой: Parquet занимает 0,7% сырого объекта, узкий NDJSON — 9,3%.

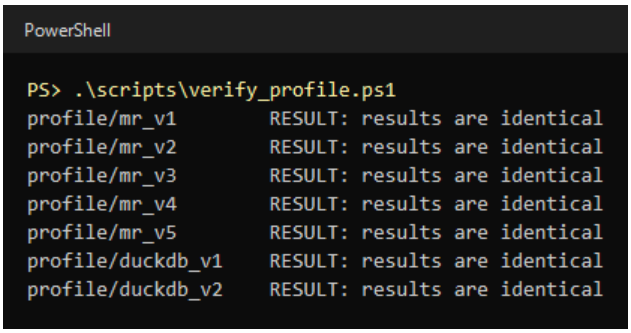


Рис. 19: Все оптимизированные версии дают результат, идентичный эталону.

Итог оптимизации. MapReduce: compute 2,10 → 0,42 с (в 5,0 раза), пик памяти 328 → 60 МиБ (в 5,4 раза). DuckDB: 0,95 → 0,24 с (в 4,0 раза), 188 → 97 МиБ (в 1,9 раза). На curated-слое собственный MapReduce отстаёт от DuckDB по времени лишь в 1,8 раза и расходует на 38% меньше памяти, потому что в нём нет буферного пула и многопоточного движка.

Неудачные эксперименты, которые тоже дали полезные выводы.

- **Открытый диапазон.** Первая версия потокового чтения запрашивала диапазон `bytes=start-` и закрывала соединение досрочно. На чанках по 0,1 МиБ (525 сплитов) это заняло 30 с: сервер начинает отдавать весь остаток объекта, и каждое досрочное закрытие обходится дорого. Поэтому итоговая версия использует ограниченный диапазон.
- **Мелкие чанки.** Даже с ограниченным диапазоном чанки по 0,1 МиБ работают 15 с против 1,2 с на 4 МиБ. Задержка одного запроса к RustFS (EC 2+2 на 9p-томах) составляет около 60 мс, поэтому на объектном хранилище выгодны немногочисленные крупные range-запросы — не меньше нескольких МиБ.

Выводы

1. RustFS развёрнут в Docker на четырёх папках-дисках. Вакансии загружены в бакет `hh-lake` одним NDJSON-объектом (10 000 вакансий, 55 МБ), туда же сохранены все результаты.
2. Собственный MapReduce читает объект из S3 range-запросами по чанкам, корректно обрабатывает строки на границах чанков (результат не зависит от размера чанка), использует combiner и стабильное партиционирование и переживает падение мапперов. Он посчитал для варианта 9 (142 вакансии) 122 навыка, 786 пар и 2 237 троек.
3. SQL-запрос DuckDB с самостоятельно написанным условием варианта дал те же числа, ноль расходящихся строк по всем трём размерам стека. Сверка поймала реальную ошибку рассинхронизации фильтров во время работы.
4. Рост числа пар и троек объяснён комбинаторикой: вакансия с k навыками даёт $\binom{k}{2}$ пар и $\binom{k}{3}$ троек. Суммы точно совпадают с результатами, три «длинные» вакансии дают 35% всех троек.
5. Зарплату повышают навыки-маркеры senior-позиций в iOS (Objective-C +25 000 руб., MVVM +20 800 руб.). Понижают её кроссплатформа (Flutter −42 500 руб.), базовые

инструменты (Git) и связка Java/SOLID, причём последнюю создают дубли одного регионального работодателя.

6. Регионы различаются уровнем оплаты (Москва 208 800, Петербург 142 500, Краснодар 78 300 руб. при одинаковой доле senior-вакансий) и стеком (нативный в Москве, React Native в Петербурге). Москва активно нанимает удалённо (47%).
7. На сырых данных DuckDB быстрее собственного MapReduce в 2,2 раза и экономнее по памяти в 1,7 раза. После оптимизации (потоковое чтение, потоки вместо процессов, curated-слой) MapReduce ускорился в 5 раз при снижении памяти в 5,4 раза, DuckDB на Parquet — в 4 раза при снижении памяти в 1,9 раза.